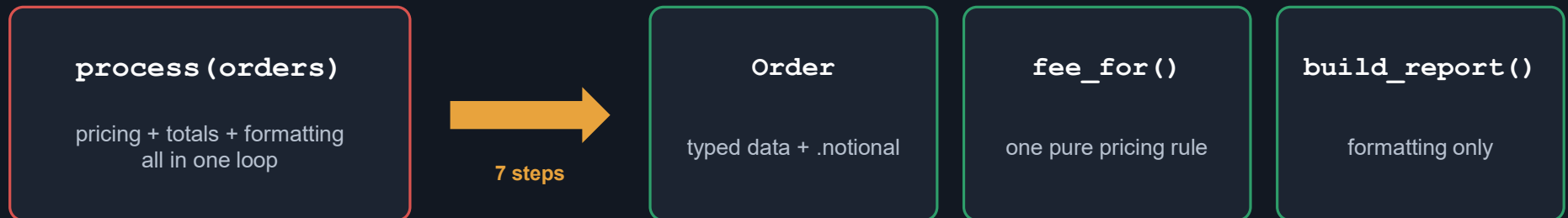


HANDS-ON REFACTORING WORKSHOP

Refactoring the Fee Report

From an 18-line legacy loop to clean, typed, testable Python — one safe, verified step at a time.



Every step is checked against the legacy output with golden-master tests.

Meet the legacy code

report_before.py — it works, it ships, and nobody wants to touch it

BEFORE report_before.py

```
1 def process(orders):
2     t = 0
3     out = []
4     for o in orders:
5         if o["type"] == "EQ":
6             f = o["qty"] * o["px"] * 0.0025
7         elif o["type"] == "FI":
8             f = o["qty"] * o["px"] * 0.0010
9         elif o["type"] == "FX":
10            f = 5.0
11        else:
12            f = 0
13        if f < 1:
14            f = 1
15        t = t + o["qty"] * o["px"] + f
16        out.append(o["id"] + "," + str(round(f, 2)))
17    out.append("TOTAL," + str(round(t, 2)))
18    return "\n".join(out)
```

The business rules hidden inside

Type	Meaning	Fee rule
EQ	Equity	0.25% of qty × px
FI	Fixed income	0.10% of qty × px
FX	Foreign exchange	Flat 5.0
other	Unknown	0
all	Minimum fee	never below 1

Try it: input → output

```
process([
    {"id": "A1", "type": "EQ", "qty": 100, "px": 50.0},
    {"id": "B2", "type": "FX", "qty": 1000, "px": 1.1},
    {"id": "C3", "type": "FI", "qty": 10, "px": 20.0}])

# A1,12.5      <- 5000 x 0.25%
# B2,5.0       <- flat FX fee
# C3,1         <- 0.2 raised to minimum
# TOTAL,6318.5 <- notional + fees
```

Diagnosis: 8 code smells in 18 lines

Each numbered marker points at where the smell lives in the code

BEFORE report_before.py

```
1  def process(orders):
2      t = 0
3      out = []
4      for o in orders:
5          if o["type"] == "EQ":
6              f = o["qty"] * o["px"] * 0.0025
7              elif o["type"] == "FI":
8                  f = o["qty"] * o["px"] * 0.0010
9              elif o["type"] == "FX":
10                 f = 5.0
11             else:
12                 f = 0
13             if f < 1:
14                 f = 1
15             t = t + o["qty"] * o["px"] + f
16             out.append(o["id"] + "," + str(round(f, 2)))
17         out.append("TOTAL," + str(round(t, 2)))
18     return "\n".join(out)
```

1 Cryptic names

process, t, f, o, out say nothing about fees or reports

2 Magic numbers

0.0025, 0.0010, 5.0, 1 — business rules with no names

3 Duplicated math

qty × px is written 3 times (L6, L8, L15)

4 If/elif chain

every new asset type means editing this function

5 Stringly-typed dicts

a typo like o["qyt"] fails only at runtime

6 Mixed responsibilities

pricing, totalling and formatting in one loop

7 Buried rule

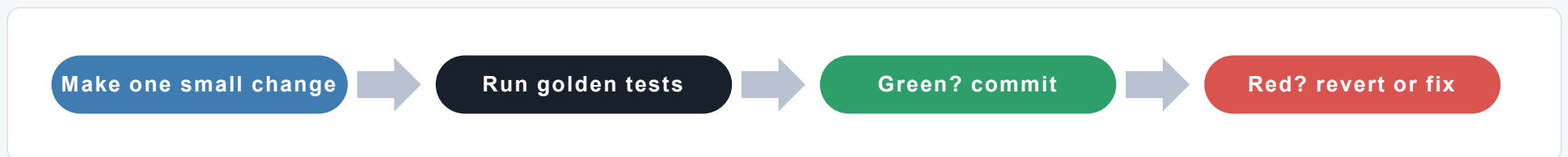
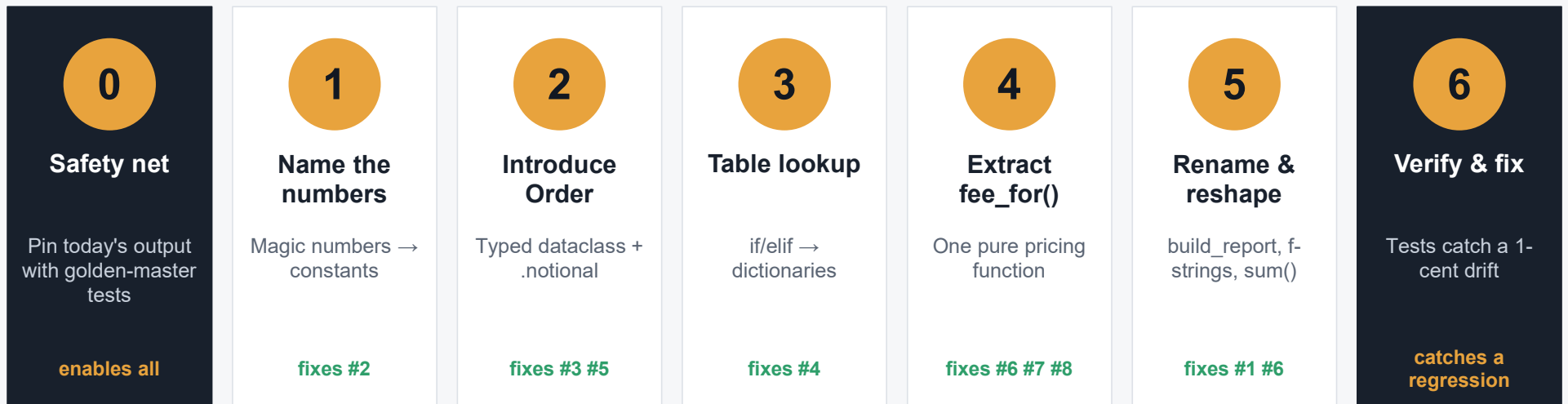
the minimum-fee clamp hides at L13–14

8 Hard to test

you can't check one fee without building a whole report

The roadmap: 7 small, reversible steps

Golden rule — change the structure, never the behaviour. Run the tests after every step.



0

Safety net: characterization tests

Before changing a single line, capture what the code does today — the legacy output is the spec

TEST

test_golden.py

```
from report_before import process as legacy # the oracle

EDGE_CASES = [
    [], # empty
    [{"id": "U", "type": "??", "qty": 1, "px": 1.0}], # unknown
    [{"id": "M", "type": "EQ", "qty": 10, "px": 10.0}], # min fee
    [{"id": "B", "type": "EQ", "qty": 400, "px": 1.0}], # fee == 1.0
    [{"id": "X", "type": "FX", "qty": 1, "px": 99.0}], # flat fee
]

def test_edge_cases():
    for orders in EDGE_CASES:
        assert new(orders) == legacy(orders)

def test_random_orders_match_legacy():
    rng = random.Random(1) # seeded = reproducible
    failures = [o for o in (random_orders(rng)
                            for _ in range(200_000))
                if new(o) != legacy(o)]
    assert not failures, f"{len(failures)} mismatches"
```



Characterization test

Doesn't ask "is it right?" — asks "is it the same?". Any output change, even a single character, fails.



Why 200,000 random cases?

The regression we meet in step 6 hits 26 of 200,000 reports (0.013%). With only 1,000 cases you'd catch it about 12% of the time.



Run it after every step

```
$ STEP=step4_extract \
  pytest -q test_golden.py
```

1

Name the magic numbers

Turn anonymous literals into named business rules

GOLDEN TESTS: PASS

BEFORE report_before.py

```
def process(orders):
    t = 0
    out = []
    for o in orders:
        if o["type"] == "EQ":
            f = o["qty"] * o["px"] * 0.0025
        elif o["type"] == "FI":
            f = o["qty"] * o["px"] * 0.0010
        elif o["type"] == "FX":
            f = 5.0
        else:
            f = 0
        if f < 1:
            f = 1
```



AFTER step1_constants.py

```
FEE_RATES = {"EQ": 0.0025, "FI": 0.0010}
FLAT_FEES = {"FX": 5.0}
MIN_FEE = 1 # int on purpose: prints "1", not "1.0"

for o in orders: # inside process()
    if o["type"] == "EQ":
        f = o["qty"] * o["px"] * FEE_RATES["EQ"]
    elif o["type"] == "FI":
        f = o["qty"] * o["px"] * FEE_RATES["FI"]
    elif o["type"] == "FX":
        f = FLAT_FEES["FX"]
    else:
        f = 0
    if f < MIN_FEE:
        f = MIN_FEE
```

1 What we did

Moved every business number into three module-level constants. Logic unchanged — only the literals were replaced.

2 Why it's better

The whole fee schedule reads at a glance, lives in one place, and the names explain what each number means.

! Watch out

Keep MIN_FEE as int 1. Writing 1.0 would print "C3,1.0" instead of "C3,1" — the golden test would catch it.

2

Introduce a typed Order

Replace raw dicts with an immutable dataclass that knows its own notional

GOLDEN TESTS: PASS

BEFORE step1_constants.py

```
def process(orders):
    t = 0
    out = []
    for o in orders:
        if o["type"] == "EQ":
            f = o["qty"] * o["px"] * FEE_RATES["EQ"]
        elif o["type"] == "FI":
            f = o["qty"] * o["px"] * FEE_RATES["FI"]
        ...
        t = t + o["qty"] * o["px"] + f
    out.append(o["id"] + "," + str(round(f, 2)))
```



AFTER step2_order.py

```
@dataclass(frozen=True)
class Order:
    id: str
    type: str
    qty: int
    px: float

    @property
    def notional(self) -> float:
        return self.qty * self.px

def process(orders: list[Order]) -> str:
    if o.type == "EQ":
        f = o.notional * FEE_RATES["EQ"]
    ...
    t = t + o.notional + f
    out.append(o.id + "," + str(round(f, 2)))
```

1 What we did

Created a frozen Order dataclass with typed fields. $\text{qty} \times \text{px}$ now lives once, in the notional property.

2 Why it's better

Typos become errors your IDE or mypy flags before running. One formula for notional. Orders can't be mutated mid-report.

! Watch out

Callers must pass Order objects now — convert at the boundary: `[Order(**d) for d in raw]`. $(\text{qty} \times \text{px}) \times \text{rate}$ keeps the legacy float order.

3

Replace the if/elif chain with lookups

Make the fee schedule data, not code

GOLDEN TESTS: PASS

BEFORE step2_order.py

```
for o in orders:
    if o.type == "EQ":
        f = o.notional * FEE_RATES["EQ"]
    elif o.type == "FI":
        f = o.notional * FEE_RATES["FI"]
    elif o.type == "FX":
        f = FLAT_FEES["FX"]
    else:
        f = 0
    if f < MIN_FEE:
        f = MIN_FEE
```



AFTER step3_lookup.py

```
for o in orders:
    if o.type in FLAT_FEES:
        f = FLAT_FEES[o.type]
    else:
        f = o.notional * FEE_RATES.get(o.type, 0.0)
    if f < MIN_FEE:
        f = MIN_FEE
```

```
# New asset type? It's a data change, not a code change:
FEE_RATES = {"EQ": 0.0025, "FI": 0.0010, "CM": 0.0015}
```

1 What we did

Swapped the 4-way branch for two dictionary lookups: flat fees first, then percentage rates with a 0.0 default.

2 Why it's better

Open/Closed principle — adding an asset type means one new dict entry. Fewer branches, fewer tests to write.

! Watch out

Unknown types now get 0.0 instead of 0. Both are raised to MIN_FEE, so the output is identical. Keep each type in only one dict.

4

Extract fee_for() as a pure function

Give the pricing rule its own name, its own home, and its own tests

GOLDEN TESTS: PASS

BEFORE step3_lookup.py

```
def process(orders: list[Order]) -> str:
    t = 0
    out = []
    for o in orders:
        if o.type in FLAT_FEES:
            f = FLAT_FEES[o.type]
        else:
            f = o.notional * FEE_RATES.get(o.type, 0.0)
        if f < MIN_FEE:
            f = MIN_FEE
        t = t + o.notional + f
    out.append(o.id + "," + str(round(f, 2)))
```



AFTER step4_extract.py

```
def fee_for(order: Order) -> float:
    if order.type in FLAT_FEES:
        fee = FLAT_FEES[order.type]
    else:
        fee = order.notional * FEE_RATES.get(order.type, 0.0)
    return max(fee, MIN_FEE)
```

```
def process(orders: list[Order]) -> str:
    ...
    for o in orders:
        f = fee_for(o)
        t = t + o.notional + f
```

```
# Now each rule is testable on its own:
assert fee_for(Order("A1", "EQ", 100, 50.0)) == 12.5
assert fee_for(Order("M", "EQ", 10, 10.0)) == 1
```

1 What we did

Moved the fee logic into `fee_for(order)`. The two-line clamp became `max(fee, MIN_FEE)`.

2 Why it's better

Single responsibility: one function, one rule. Unit-test fees without building reports; reuse it anywhere (e.g. a quote API).

! Watch out

Argument order matters! On a tie `max()` returns the first argument: `max(1.0, 1) → 1.0` prints "1.0" like legacy. `max(MIN_FEE, fee)` would print "1".

5

Rename, then split calc from format

Readable names, a comprehension for lines, sum() for the total

GOLDEN TESTS: FAIL

BEFORE step4_extract.py

```
def process(orders: list[Order]) -> str:
    t = 0
    out = []
    for o in orders:
        f = fee_for(o)
        t = t + o.notional + f
        out.append(o.id + "," + str(round(f, 2)))
    out.append("TOTAL," + str(round(t, 2)))
    return "\n".join(out)
```



AFTER report_after.py

```
def build_report(orders: list[Order]) -> str:
    lines = [f"{o.id},{round(fee_for(o), 2)}"
              for o in orders]
    total = sum(o.notional + fee_for(o)
                 for o in orders)
    lines.append(f"TOTAL,{round(total, 2)}")
    return "\n".join(lines)
```

```
# $ STEP=step5_after pytest -q test_golden.py
# FAILED 26 mismatches, e.g. TOTAL,3009489.5
#      (legacy says TOTAL,3009489.51)
```

1 What we did

Renamed process → build_report, t/f/out → total/fee/lines. Lines via a list comprehension, total via sum(), f-strings for output.

2 Why it's better

The code now reads like the spec: "one line per order, then a total". This is the report_after.py you were given.

! Watch out

The golden test goes RED: 26 of 200,000 reports are off by one cent. It looks equivalent — it isn't. Next: find out why.

6

Diagnose: floats aren't associative

The smallest failing case the random test found — two orders, one cent apart

GOLDEN TESTS: FAIL

REPRO

```
orders = [Order("00", "FX", 4942, 208.4919),
           Order("01", "EQ", 3952, 499.54)]
```

Legacy loop

```
t = (t + notional)
    + fee
```

```
3009489.5050000004
```

...489.51

report_after.py

```
total += (notional
          + fee)
```

```
3009489.505
```

...489.5

Same numbers, different grouping → different last bit → the value lands on the other side of the .505 rounding boundary.

PYTHON REPL

```
>>> (0.1 + 0.2) + 0.3
0.6000000000000001
>>> 0.1 + (0.2 + 0.3)
0.6
>>> round(3009489.5050000004, 2)
3009489.51
>>> round(3009489.505, 2)
3009489.5
```

! Root cause

Legacy adds notional to the running total, then the fee.
`sum(o.notional + fee_for(o))` adds notional + fee first, then the total.
 In floats, $(a+b)+c \neq a+(b+c)$. Bonus smell: `fee_for()` is now called twice per order.

6

Fix: keep the shape, restore the order

One loop, one `fee_for()` call per order, same addition order as before

GOLDEN TESTS: PASS

FIXED

step6_fixed.py

```
def build_report(orders: list[Order]) -> str:
    lines, total = [], 0
    for o in orders:
        fee = fee_for(o)                # computed once
        total = total + o.notional + fee # legacy float order
        lines.append(f"{o.id},{round(fee, 2)}")
    lines.append(f"TOTAL,{round(total, 2)}")
    return "\n".join(lines)
```

TERMINAL

```
$ STEP=step5_after pytest -q test_golden.py
1 failed, 1 passed

$ STEP=step6_fixed pytest -q test_golden.py
2 passed
```

Two legitimate ways forward

A. Pure refactor (shown)

Keep behaviour byte-identical. Match the legacy summation order. Ship with zero business sign-off needed.

B. Deliberate fix, separate change

Money in floats is the real bug. Switch to Decimal in its own commit, agree the new numbers with the business, then re-record the golden files.

Edge cases the refactor must preserve

Subtle int/float details in the legacy output that the new code reproduces on purpose

Scenario	Input	Legacy output	Final code	Why it matches
Empty list	<code>[]</code>	<code>TOTAL, 0</code>	match	Running total starts at int 0, so <code>round(0, 2)</code> prints "0"
Unknown type	<code>?? 1 × 1.0</code>	<code>U, 1 / TOTAL, 2.0</code>	match	<code>.get(type, 0.0) → max(0.0, 1)</code> returns the int 1
Below minimum	<code>EQ 10 × 10.0</code>	<code>M, 1</code>	match	fee 0.25 → <code>max(0.25, 1)</code> returns the int 1
Exactly minimum	<code>EQ 400 × 1.0</code>	<code>B, 1.0</code>	match	Tie: <code>max(1.0, 1)</code> keeps the first argument, the float 1.0
Flat FX fee	<code>FX 1 × 99.0</code>	<code>X, 5.0</code>	match	FLAT_FEES is checked before the rate table
Large mixed orders	<code>FX + EQ (above)</code>	<code>TOTAL, 3009489.51</code>	match after fix	<code>report_after.py</code> printed ...489.5 — fixed by restoring the summation order

Lesson: output formats are contracts. "1" vs "1.0" and ".51" vs ".5" would break any downstream parser or reconciliation that diffs these reports.

Before vs after: the scorecard

Slightly more lines — far less to hold in your head

Aspect	Before	After
Named units	1 function	Order, fee_for, build_report
Magic numbers in logic	4	0
qty × px written	3 times	once (.notional)
Fee branches	4-way if/elif	1 if + lookups
New asset type	edit the loop	add a dict entry
Input safety	dict keys, runtime KeyError	typed, frozen dataclass
Unit-testable fee	no	yes
Output	baseline	identical on 200,005 cases

18 → 34 lines, but every piece now has one job and a name that says what it is.

FINAL step6_fixed.py

```
FEE_RATES = {"EQ": 0.0025, "FI": 0.0010}
FLAT_FEES = {"FX": 5.0}
MIN_FEE = 1 # int on purpose: legacy output prints "1", not "1.0"

@dataclass(frozen=True)
class Order:
    id: str
    type: str
    qty: int
    px: float
    @property
    def notional(self) -> float:
        return self.qty * self.px

def fee_for(order: Order) -> float:
    if order.type in FLAT_FEES:
        fee = FLAT_FEES[order.type]
    else:
        fee = order.notional * FEE_RATES.get(order.type, 0.0)
    return max(fee, MIN_FEE)

def build_report(orders: list[Order]) -> str:
    lines, total = [], 0
    for o in orders:
        fee = fee_for(o)
        total = total + o.notional + fee # keep legacy float order
        lines.append(f"{o.id},{round(fee, 2)}")
    lines.append(f"TOTAL,{round(total, 2)}")
    return "\n".join(lines)
```

Your turn: hands-on lab

Use the step files and `test_golden.py` that come with this deck

LAB 1

EASY

Add commodities

Charge CM orders 0.15% of notional.

Touch only `FEE_RATES`. Add a `fee_for()` unit test. Golden tests stay green (legacy never saw CM, so add a new expected case).

LAB 2

MEDIUM

Cap the equity fee

EQ fees can't exceed 500.

Where does the rule belong? (`fee_for`). This changes behaviour on purpose — write the new test first, and expect some golden cases to change.

LAB 3

HARD

Money as Decimal

Remove float rounding surprises.

Do it as a separate, deliberate change. Decide the rounding mode (`ROUND_HALF_UP?`), get sign-off, then re-record the golden output.

```
steps/
  report_before.py      # legacy oracle
  step1_constants.py    step2_order.py      step3_lookup.py
  step4_extract.py      step5_after.py     step6_fixed.py
  test_golden.py        # STEP=<module> pytest -q test_golden.py
```

Six habits to take back to your codebase

1 Pin behaviour first

Golden-master tests turn "I think it's the same" into proof.

2 Take small steps

One idea per step, tests after each. Red means revert, not debug for hours.

3 Name everything

Constants, types and functions are documentation that can't go stale.

4 Prefer data to branches

A lookup table grows by adding rows, not by editing logic.

5 Split calc from format

Pure functions for rules, a thin layer for output.

6 Trust tests over eyes

sum() looked equivalent. Floats disagreed. Only the test knew.